

1. Omów mechanizm journalingu. Wymień nazwy systemów plików, w których jest stosowany.

Przy użyciu księgowania dane nie są od razu zapisywane na dysk, tylko zapisywane w swoistym dzienniku/kronice (ang. journal). Dzięki takiemu mechanizmowi działania zmniejsza się prawdopodobieństwo utraty danych: jeśli utrata zasilania nastąpiła w trakcie zapisu - zapis zostanie dokończony po przywróceniu zasilania, jeśli przed - stracimy tylko ostatnio naniesione poprawki, a oryginalny plik pozostanie nietknięty. De facto księgowanie jest formą realizacji zapisu transakcyjnego - jaki zwykle oferowały transakcyjne bazy danych (DB2, IMS, Oracle, RDB i inne), czy serwery transakcyjne (TTS, CICS).

Systemami plików z księgowaniem są: ext3, ext4, NTFS, HFS+ (system plików Apple dla Mac OS).

2. Omów proces uruchamiania komputera z BIOSem (od wciśnięcia przycisku zasilania do załadowania systemu operacyjnego).

- BIOS POST – pierwsza wykonana czynność. Zazwyczaj testuje integralność sprzętu (rejstry, pamięć, klawiatura etc.) i wyłącza niemaskowalne przerwanie.
- Załadowanie MBR – Master Boot Record – pierwszy sektor dysku twardego zawierający program rozruchowy
- Załadowanie VBR – Volume Boot Record – jak wyżej, tyle, że na partycjach
- odpalenie boot menagera – umożliwia załadowanie OS (wyświetla też menu wyboru)
- Przetworzenie BCD – Boot Configuration Data – dane do użycia w trakcie bootowania
- winload.exe – bootloader windowsa przejmuje pałeczkę, on ładuje wszystko aż do sterowników boot-class
- ładowanie SYSTEM hive – zawiera ustawienia nie przyporządkowane do żadnego użytkownika
- samosprawdzenie integralności winload.exe
- załadowanie i sprawdzenie ntoskrnl.exe, hal.dll – jądro systemu i Hardware Abstraction Layer
- ładowanie, weryfikacja i inicjalizacja importów ntoskrnl.exe
- skanowanie rejestrów
- ładowanie, weryfikacja i inicjalizacja sterowników boot-class
- ntoskrnl.exe jest inicjalizowany – w końcu. Przejmuje pałeczkę.
- inicjalizacja podsystemów, zbudowanie struktury systemu
- skanowanie rejestrów
- ładowanie, weryfikacja i inicjalizacja sterowników i usług „system class”
- smss.exe – Session Manager Subsystem – tworzy zmienne środowiskowe i generalnie przejmuje pałeczkę od tego momentu
- odpalenie aplikacji startupowych określonych przez wpis BootExecute w rejestrze
- załadowanie jądra podsystemu Win32K.sys
- załadowanie znanych DLL'ek
- podział:
 - Session 0 smss.exe odpala
 - o csrss.exe – client-server run-time subsystem – odpowiada za wątki m.in
 - o wininit.exe – program inicjalizujący windowsa
 - o lsass.exe, lsm.exe, services.exe -jakieś tam rzeczy dla windowsa
 - Session 1 smss.exe odpala
 - o csrss.exe - client-server run-time subsystem – odpowiada za wątki m.in
 - o winlogon.exe - wiadomo
 - o logonUI.exe – jeszcze bardziej wiadomo
 - o userinit.exe – program inicjalizujący usera

3. Przedstaw charakterystykę systemów opartych o EFI.

EFI: Extensible Firmware Interface - definiuje interfejs pomiędzy systemem operacyjnym a firmwarem sprzętu, następcą BIOS.

Systemy EFI:

- są niezależne od implementacji sprzętu

- nie wymagają trybu rzeczywistego procesora do rozruchu
- wykorzystują płaski model pamięci, gdzie cała dostępna pamięć może być zaadresowana
- nie nakładają limitów na całkowity rozmiar pamięci ROM. Sterowniki i aplikacje mogą być ładowane z dowolnego źródła w dowolne miejsce przestrzeni adresowej.
- Zawierają dodatkową powłokę (EFI shell)
- Ewoluuja od VGA do UGA (Universal Graphics Adapter)
- Jako pierwsza załadowana zostaje cienka warstwa PEI, która wykonuje większość rzeczy związanych z rozruchem komputera (inicjalizacja chipsetu, pamięci, enumeracja szyn danych etc.)
- Następnie EFI przygotowuje DXE (Driver Execution Environment) by zapewnić podstawowe funkcje, których mogą użyć sterowniki EFI
- Wykrywane są partycje
- Następuje uruchomienie bootloadera i jak wszystko ok, to przekazujemy sterowanie do OSA. W przeciwnym wypadku uruchamiany jest EFI Shell.
- Umożliwiają bootowanie z dużych dysków (>2TB)
- Posiada własny boot loader
- Szybszy proces bootowania
- Wsparcie dla sieci, bardziej przyjazny interfejs graficzny.

4. Wyjaśnij różnicę między liniowym a fizycznym adresem w pamięci operacyjnej.

Adres fizyczny jest adresem bezpośrednio określającym położenie segmentu w pamięci, przez ten adres maszyna musi się do odpowiedniego segmentu odwoływać. Adres liniowy natomiast składa się z numeru katalogu (10-bit), odnoszącego się do Directory Entry w Page Directory, numeru tabeli (10-bit), odnoszącego się do Table Entry w Page Table i offsetu (12-bit). Taki adres, zapisany w Dekryptorze Segmentu w Global Descriptor Table może zostać wykorzystany przez sprzęt dopiero po przetłumaczeniu z użyciem wyżej wymienionych stron na adres fizyczny.

W przypadku wyłączenia stronicowania, adres liniowy jest tożsamy adresowi fizycznemu wyznaczonemu w opisany sposób.

5. Wymień metody ukrywania danych (plików) w systemach plików i opisz pokrótce ideę każdej z nich:

Out of band: zapis w niewykorzystanym i niewidocznym przez system miejscu. Wynika z rozmiaru klastra - pojemność dysku jest skokowa i przy partycjonowaniu może nam zostać trochę nieużywanego miejsca.

In-band: zapis w miejscu zarezerwowanym dla pliku - np. klaster 4kb, plik 1kb i zostaje wolne 3kb (plik nawet jednobitowy zajmuje na dysku cały klaster)

Ukrywanie partycji: (np. w laptopach partycja do odzyskiwania systemu) typ partycji ustawiamy na jakiś swój i wtedy system nie wie co to jest i zlewa.

Ukrywanie w warstwie aplikacyjnej: pewnie ukrywamy jakieś dane w obszarze, gdzie niby aplikacja ma być.

6. Podaj cele stosowania struktury GDT.

- uzyskanie liniowej przestrzeni pamięci, niezależnie od fizycznego rozproszenia danych
- umożliwienie obsługi większych ilości pamięci niż przy adresowaniu fizycznym
- zwiększenie funkcjonalności i kontroli nad tym, co się dzieje - dodanie opisów segmentów - kod/dane, opisy przywilejów, czy segment jest w pamięci
- zwiększenie bezpieczeństwa

9. Jaką rolę w systemie plików NTFS pełni nadrzędna tablica plików (MFT)?

Master File Table (MFT) - metaplik zawiera wpisy (rekordy plikowe) opisujące każdy plik/katalog

znajdujący się na partycji NTFS w postaci rekordu plikowego (1024 - 4096 bajtów). Rekord plikowy składa się z nagłówka oraz sekwencji występujących po sobie atrybutów w postaci pary (atrybut, wartość). Charakterystycznym elementem nagłówka rekordu plikowego są znaki „FILE” (w kodzie

ASCII) umieszczone na początku nagłówka. Nagłówek zawiera kilkanaście pól o różnym przeznaczeniu, m.in. podany jest faktyczny rozmiar rekordu plikowego (4-bajtowe pole o offsecie 18H) oraz zaalokowany rozmiar rekordu plikowego (4-bajtowe pole o offsecie 1CH).

10. Budowa i rola rekordu plikowego w NTFS.

Atrybuty w rekordzie plikowym:

- Informacje standardowe – znaczniki czasu, ilość łączy, prawa dostępu
- Nazwa pliku/katalogu
- Informacje o woluminie – nazwa woluminu
- Dane – dane pliku
- Indeks główny – używany do implementowania katalogów i indeksów
- Mapa bitowa – dane tego atrybutu zawierają pole bitowe

Rodzaje atrybutów:

- rezydentne - ich rozmiar nie przekracza rozmiaru rekordu plikowego
- nierezydentne - większe od rozmiaru rekordu plikowego, wówczas w miejscu danych znajduje się lista wskaźników do kolejnych części danych.

11. W jaki sposób tablica alokacji plików (FAT) opisuje położenie plików na dysku?

File allocation table (FAT) zawiera informacje o stanie wszystkich klastrów obszaru danych dysku.

Każda pozycja

tablicy używana przez plik wskazuje następną pozycję w FAT dla tego pliku (metoda listowa).

Sekwencja numerów kolejnych pozycji jest taka sama jak numery wiązek składających się na cały plik.

Pierwsze dwie pozycje w tablicy są zarezerwowane. Trzecia zawiera informację o pierwszym klastrze obszaru danych (numer 2). Każdy wpis ma 32 bajty i zawiera m.in:

- krótką nazwę pliku
- czas i datę modyfikacji/utworzenia/dostępu
- atrybuty pliku
- rozmiar pliku

12. Omówić strukturę partycji rozszerzonej na dysku.

Pojedyncza tablica partycji może pomieścić informacje tylko o 4 dyskach logicznych. Celem zwiększenia tej liczby zdefiniowano partycję rozszerzoną w skład, której może wchodzić wiele dysków logicznych, które przez użytkownika są widziane, jako zwykłe dyski. Partycje rozszerzone są stosowane w systemie Windows. Na początku partycji rozszerzonej znajduje się sektor o specyficznej strukturze, w którym zawarta jest tablica partycji, położona w tym samym miejscu, co w przypadku MBR'a. W tablicy tej wykorzystywane są tylko dwie pierwsze pozycje. Pierwszy element tablicy partycji wskazuje adres początkowy dysku logicznego. Drugi – sektor dyskowy o takiej samej strukturze, stanowiący kolejny element listy dysków logicznych. Element zawierający 0 w polu identyfikatora systemu operacyjnego druga kolumna wskazuje koniec tej listy.

13. Opisz mechanizm obsługi przerwania na poziomie jądra w systemie Windows XP. Wyjaśnij rolę zmiennej IRQL.

Przerwania w systemie Windows są relizowane poprzez wirtualizację obsługi przerwania (przerwania od sprzętu są dostarczane w odpowiedniej formie z warstwy HAL). Przerwania mogą mieć różny priorytet (określa to zmienna IRQL - Interrupt Request Level), przerwanie o niższym priorytecie mogą zostać przerwane (zamaskowane) poprzez przerwanie o wyższej wartości IRQL.

Kiedy wystąpi przerwanie, jest ono obsługiwane przez funkcję ISR (Interrupt Service Routine). Struktura IDT (Interrupt Dispatch Table - związana z konkretnym procesorem) kontroluje która funkcja obsłuży przerwanie z danym IRQL. Sterowniki łączą ISR z konkretnym IRQL poprzez stworzenie obiektu przerwania (interrupt object) i przekazując go do kernela. Łączy go on z odpowiednim wpisem w tablicy IDT (z każdym IRQL może związane być więcej niż 1 przerwanie), w przypadku odpięcia sterownika(urządzenia) obiekt przerwania jest po prostu usuwany z tablicy.

18. Omówić implementację pamięci wirtualnej za pomocą segmentacji.

Polega na przechowywaniu niektórych segmentów programu w pamięci dyskowej i sprowadzaniu ich do pamięci operacyjnej, gdy stają się potrzebne.

Obecność segmentu w pamięci operacyjnej wskazuje bit P(present). Jeżeli wystąpi odwołanie do deskryptora z $P = 0$ to pojawia się wyjątek(przerwanie procesora), w takiej sytuacji system odszukuje wolny obszar pamięci (lub ją zwalnia), pobiera potrzebny segment do pamięci i powraca do realizowanego programu. Aby racjonalnie zorganizować wymianę segmentów potrzeba mieć informację o statystykach wykorzystania poszczególnych segmentów, do zbierania informacji pomocny jest bit A(access). Ustawiany jest on gdy do rejestru segmentowego zostanie wpisany selektor wskazujący rozpatrywany deskryptor. System operacyjny (nie procesor) zeruje go w obserwowanych segmentach oraz okresowo odczytuje jego wartość i wykonuje zerowanie. Jeżeli odczytano kilkakrotnie 0, znaczy to iż segment jest nieużywany. Istotny w implementacji jest również bit W, system może początkowo przypisać wszystkim segmentom atrybut „tylko do odczytu”(W=0), każda próba zapisu do danego segmentu wywoła wyjątek, może on sprawdzać „legalność” operacji i jeżeli zdecydujemy się na zapis ustawiamy bit W. Daje to informacje o tym iż zawartość segmentu została zmieniona – jeżeli istnieje potrzeba usunięcia go z pamięci należy go przepisać na dysk, w przeciwnym wypadku zawartość segmentu w pamięci i na dysku jest jednakowa i nie ma potrzeby kopiowania go.

19. Jakie zalety ma realizacja pamięci wirtualnej za pomocą stronicowania w porównaniu z realizacją za pomocą segmentacji?

Segmentacja:

- deskryptor zawiera adres bazowy i rozmiar.
- segmenty różnych rozmiarów i słabo to systemowi przerzucać, pamięć się fragmentuje

Stronicowanie:

- zamiast adresy bazowego wpis w katalogu stron i indeks wpisu (deskryptor = adres fizyczny katalogu stron + indeks w tablicy stron + offset)
- odwołanie przez katalog stron -> tablica stron -> adres fizyczny (działa jak segregator z zakładkami segregator -> zakładka -> strona)
- każdy procesor ma swój katalog stron (segregator) w rejestrze CR3
- w deskryptorze bit P - czy segment jest w pamięci -> zwalnianie ostatnio nieużywanych stron i ładowanie potrzebnych z pamięci. Proces odwołując się do ramki wstawia tam 1, system operacyjny go zeruje i robi sobie jakieś statystyki.
- rejestr cieni - procesor zapisuje sobie adresy rzeczywiste ramek, żeby nie robić za każdy razem tej translacji.
- tablica ostatnich adresów - używana jeśli nowy adres różni się tylko offsetem
- segmenty składają się ze stałego rozmiaru, zazwyczaj 4kb. Łatwiej jest systemowi zarządzać wieloma małymi kawałkami niż jednym dużym.

Segmentacja powoduje fragmentację pamięci (uboczny skutek wymiany segmentów o różnych rozmiarach), co nie ma miejsca przy stronicowaniu.

Poszczególne procesy mają dostęp do wyłącznie swoich własnych obszarów pamięci (odwołania przechodzą przez tablicę transformacji adresów i kierowane są pod obszar pamięci procesu).

29. Ile stosów ma procedura w trybie chronionym? Wyjaśnij dlaczego tak się dzieje.

Dana procedura wykonywana jest przez wątek. Każdy wątek posiada dwa stosy – po jednym w trybie jądra i użytkownika. Powodem tego jest bezpieczeństwo – nic, co działa w trybie użytkownika nie ma dostępu do danych z trybu jądra.

32. Omów typowe działanie systemu operacyjnego przy błędach strony (page fault handling).

Błąd strony jest zgłaszany przez sprzęt w czasie, gdy program uzyskuje dostęp do strony, który jest mapowany w wirtualnej przestrzeni adresowej, ale nie załadowany do pamięci fizycznej. Systemy

operacyjne takie jak Windows i UNIX (i inne systemy uniksowe) zapewniają różne mechanizmy raportowania błędów spowodowanych przez błędy strony. Windows w takim generuje wyjątek, natomiast UNIX (i uniksopodobne) zazwyczaj używają sygnałów, takie jak SIGSEGV, do zgłaszania tych błędów w programach. Jeśli program, który otrzymał błąd nie obsługuje go, to system operacyjny zazwyczaj wykonuje jakieś działanie domyślne (komunikat dla użytkownika oraz zrzut pamięci).

38. Wyjaśnij następujące pojęcia MBR, VBR, DDK, IDT, GDTR.

MBR – Master Boot Record. Jest to pierwszy sektor na dysku zawierający specjalny program, który jest odczytywany przy starcie komputera do pamięci RAM i którego zadaniem jest przeszukanie tablicy partycji w celu przejścia do VBR.

VBR – Volume Boot Record. Podobny do MBR, ale znajduje się już na konkretnej partycji (na początku) i zawiera program, który powinien rozpocząć ładowanie systemu operacyjnego.

DDK – Driver Development Kit. Zestaw narzędzi pozwalających na tworzenie sterowników urządzeń (i nie tylko) na pewną platformę.

IDT – Interrupt Descriptor Table. Jest to tablica deskryptorów znajdująca się w pamięci RAM, w której zapisane są selektory, które z kolei mają indeksować tablicę GDT, która wskazuje segment, w jakim znajduje się procedura obsługi przerwania.

GDTR – Global Descriptor Table Register. Jest to 48-bitowy rejestr wskazujący położenie w pamięci tablicy GDT (32 bity) oraz limit rozmiaru tej tablicy (16 bitów).

39. Wymień 5 różnych systemów plików.

fat32 - listowy plik do 4GB, dysk do 2TB

reiserfs - journaling, wydajny dla małych plików,

ntfs - hybryda listowej i indeksowej metody (lista kolejnych segmentów pliku zawierających rozmiar ciągłego obszaru

zajmowanego przez fragment pliku)

xfs - 64bitowy gwarantowanie pasma dostępu do pliku

ext3 - metoda indeksowa, journaling, można montować jako ext2(narzędzie tune2fs), do 32TB, mniej obciąża procesor

niz reiser i xfs

42. Opisz strukturę bio zdefiniowaną w <linux/bio.h>.

Reprezentuje aktywne operacje IO w postaci listy ciągłych obszarów w pamięci (segmentów). Bufory nie muszą zajmować ciągłej przestrzeni w pamięci (scatter-gather I/O). Głównymi elementami struktury są:

- o bio_io_vec - tablica wektorów bio_vec - opisuje ona buffor.

- o bio_vec - wektor opisujący segmenty.

- o Informacje o urządzeniu do którego się odnosi

- o Informacje o typie operacji (read/write)

- o licznik użyc Struktura nie zawiera informacji o stanie buffora.

43. Porównaj poznane schedulery dla urządzeń blokowych I/O w systemie Linux.

Linus Elevator

W momencie pojawienia się nowego żądania możliwe są 4 operacje:

- gdy w kolejce znajduje się żądanie do przyległego sektora dysku, żądania te są scalane,

- gdy żądanie znajdujące się w kolejce jest w niej już długo, nowe żądanie jest umieszczane na końcu kolejki – w celu uniknięcia zagłodzenia starszego żądania,

- gdy w kolejce znajdują się żądania do bliskich sektorów nieprzylegających do siebie ani do sektorów obsługiwanych przez nowe żądanie – jest ono umieszczane pomiędzy tymi żądaniami z kolejki

- gdy nie zachodzą powyższe reguły, żądanie umieszczane jest na końcu kolejki

Deadline I/O scheduler

Każde żądanie jest oznaczone czasem wygaśnięcia (500 milisekund dla operacji odczytu, 5 sekund dla operacji zapisu). Poza posortowaną wg kolejności sektorów kolejką dodatkowa implementacja dwóch kolejek FIFO (dla odczytu oraz dla zapisu).

Anticipatory I/O scheduler

Heurystycznie przewiduje operacje, które mogą zaistnieć na podstawie statystyk dla każdego z procesów.

CFQ I/O scheduler

Domyślny scheduler dla kernela 2.6 – osobna kolejka żądań dla każdego procesu, pobieranie kolejnych żądań algorytmem round-robin.

Noop I/O scheduler

Dobry dla urządzeń o losowym dostępie (np. kart pamięci flash) brak sortowania.

44. Czym charakteryzuje się problem „write starving reads” i gdzie występuje?

Problem związany z kolejkowaniem żądań IO przez linux elevator (i nie tylko) (domyślny dla jądra 2.4).

Zapis asynchroniczny, odczyt synchroniczny.

Zapisy w jednej części dysku blokują odczyty w innej (aplikacja czeka na dane).

Deadline zapobiega częściowo zapobiega temu problemowi a anticipatory robi to świetnie.

45. Przedstaw strukturę i-node dla systemu plików ext3.

Każdy i-węzeł zawiera:

- Opis położenia pliku na dysku i rozmiar pliku
- Identyfikator właściciela pliku,
- Typ pliku (pliki zwykłe, katalogi, ...),
- Prawa dostępu do pliku (właściciel pliku, grupowy właściciel pliku, pozostali użytkownicy),
- Czas dostępu do pliku (czas ostatniej modyfikacji, czas ostatniego dostępu, czas ostatniej modyfikacji i-węzła pliku),
- Liczba dowiązań do pliku,
- Liczba procesów, które otworzyły dany plik,
- Inne,
- można zauważyć, że i-węzeł nie zawiera nazwy pliku

46. Podaj blok oraz offset, gdzie znajduje się bajt 6 000 000 w systemie plików ext3. Przedstaw tok obliczania.

bezpośrednie - do 48kb

pośrednie raz - 4mb

pośrednie dwa razy - 4gb

pośrednie trzy razy 4tb

Bajt 6 000 000 wymaga użycia bloku podwójnie pośredniego:

- Pierwszy bajt dostępny jest poprzez blok podwójnie pośredni i ma numer

$$48KB + 4MB = 49\,152 + 4\,194\,304 = 4\,243\,456$$

- Bajt o numerze 6 000 000 jest więc bajtem o numerze 1 756 544 względem początku bloku podwójnie

pośredniego

- Każdy blok pojedynczo pośredni daje dostęp do 4MB, więc bajt o numerze 6 000 000 będzie wskazywany

pośredni przez pozycję zerową bloku podwójnie pośredniego – po odczytaniu wyznaczonego w ten sposób bloku

pośredniego poszukuje się w nim pozycji o indeksie $1\ 756\ 544 / 4096 = 428$ (reszta 3456),
– Zawartość pozycji o indeksie 428 wskazuje na blok dyskowy zawierający żądany bajt – jest on
odległy o 3456
bajtów od początku bloku.

51. Zaprezentuj schematycznie i opisz budowę kernela w systemie Linux.

- nie posiada dostępu do biblioteki C ani do standardowych nagłówków języka C, zaimplementowany w GNU C (gcc, Intel C),
- Jądro monolityczne+moduly
- brak ochrony pamięci, (to chyba nie jest do końca prawda)
- posiada trudności z wykonywaniem operacji zmiennoprzecinkowych,
- dla każdego procesu przeznacza taki sam relatywnie mały stos (x86 – konfigurowalny podczas kompilacji – 4KB lub 8KB – historycznie)
- stos kernela zawiera 2 strony - 8KB na 32 bitowych – 16KB na 64 bitowych architekturach),
- podatny na wyścigi - synchronizacja oraz współbieżność (musi obsługiwać asynchronicznie nadchodzące przerwania, możliwe jest wywłaszczenie procesu, wspiera SMP),
- powinien zapewniać przenośność na różne architektury,
- ostatnia stabilna wersja Linux kernel 2.6.37 (19.01.11)